

lux Connection-Lease Reaping — Fidelity Control (Destructive Read)

Buggy variant: a read sweeps a lapsed connection without releasing it

September 2026

1 Introduction

This is the fidelity control for H1 in `docs/connection_lease_reaping.tex`, not the design. It removes only `Read`'s non-destructiveness: a lapsed connection is still correctly dropped from `registered` the moment some caller reads through it, but every scene it owned is left exactly as it was – matching `HubClientRegistry.named_sessions`, which calls `_sweep_locked` on every read and discards the swept set the moment it returns, because none of its six callers (`HubReads.client_sessions`, `CallbackMenu.from_named`, `client_details.py`, `callback_hold.py`, and the rest) forwards that set to `OwnerTracker`. Read the main specification first; this document states only what differs, in Section 6.1.

2 Basic Types

$[CONN, IDENT, SCENE]$

Unchanged from the main specification.

3 Free Types

$TSTATE ::= tup \mid tdown$

$LSTATE ::= llive \mid llapsed$

$OWNER ::= unowned \mid conn\langle\langle CONN \rangle\rangle$

Unchanged from the main specification.

4 State

ConnReg

$registered : \mathbb{P} CONN$
 $identity : CONN \leftrightarrow IDENT$
 $transport : CONN \leftrightarrow TSTATE$
 $lease : CONN \leftrightarrow LSTATE$
 $owner : SCENE \rightarrow OWNER$

$registered \subseteq \text{dom } identity$
 $\text{dom } identity = \text{dom } transport$
 $\text{dom } transport = \text{dom } lease$

Unchanged from the main specification.

5 Initialization

<i>Init</i>
<i>ConnReg'</i>
$registered' = \emptyset$
$identity' = \emptyset$
$transport' = \emptyset$
$lease' = \emptyset$
$owner' = (\lambda s : SCENE \bullet unowned)$

Unchanged from the main specification.

6 Operations

6.1 Read — BUGGY: sweeps without releasing

This is the one operation that differs from the design. A lapsed connection is still correctly dropped from **registered** – the sweep itself is not the defect, and **Depart/Claim**'s release mechanisms are deliberately left intact by this variant, isolating the H1 finding from H2 and H3. But this happens from a query, not the coordinator, and **owner** is left completely unchanged: no scene the swept connection owned is released, and because the connection is now outside **registered**, neither **Depart** nor **Claim**'s sweep can ever find it again to release it later.

<i>Read</i>
$\Delta ConnReg$
$c? : CONN$
$c? \in registered$
$lease\ c? = llapsed$
$registered' = registered \setminus \{c?\}$
$owner' = owner$
$identity' = identity$
$transport' = transport$
$lease' = lease$

6.2 Connect, Renew, TransportDies, LeaseLapse, Depart, TimedReap, Claim — unchanged

Identical to the main specification: only **Read** is no longer a pure query.

<i>Connect</i>
$\Delta ConnReg$
$c? : CONN$
$i? : IDENT$
$registered' = registered \cup \{c?\}$
$identity' = identity \oplus \{c? \mapsto i?\}$
$transport' = transport \oplus \{c? \mapsto tup\}$
$lease' = lease \oplus \{c? \mapsto llive\}$
$owner' = owner$

Renew

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $lease' = lease \oplus \{c? \mapsto llive\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $owner' = owner$

TransportDies

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $transport\ c? = tup$
 $transport' = transport \oplus \{c? \mapsto tdown\}$
 $registered' = registered$
 $identity' = identity$
 $lease' = lease$
 $owner' = owner$

LeaseLapse

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $lease\ c? = llive$
 $lease' = lease \oplus \{c? \mapsto llapsed\}$
 $registered' = registered$
 $identity' = identity$
 $transport' = transport$
 $owner' = owner$

Depart

$\Delta ConnReg$
 $c? : CONN$

$c? \in registered$
 $registered' = registered \setminus \{c?\}$
 $owner' = owner \oplus$
 $(\{s : SCENE \mid owner\ s = conn(c?)\} \times \{unowned\})$
 $identity' = identity$
 $transport' = transport$
 $lease' = lease$

TimedReap $\Delta \text{ConnReg}$ $c? : \text{CONN}$
$c? \in \text{registered}$ $\text{lease } c? = \text{lapsed}$ $\text{registered}' = \text{registered} \setminus \{c?\}$ $\text{owner}' = \text{owner} \oplus$ $(\{s : \text{SCENE} \mid \text{owner } s = \text{conn}(c?)\} \times \{\text{unowned}\})$ $\text{identity}' = \text{identity}$ $\text{transport}' = \text{transport}$ $\text{lease}' = \text{lease}$
Claim $\Delta \text{ConnReg}$ $c? : \text{CONN}$ $s? : \text{SCENE}$
$c? \in \text{registered}$ $\text{transport } c? = \text{tup}$ $\text{lease}' = \text{lease} \oplus \{c? \mapsto \text{llive}\}$ $\text{registered}' = \text{registered} \setminus$ $\{c2 : \text{CONN} \mid c2 \in \text{registered} \wedge c2 \neq c? \wedge \text{lease } c2 = \text{lapsed}\}$ $(\text{owner} \oplus (\{s : \text{SCENE} \mid \exists c2 : \text{CONN} \bullet$ $c2 \in \text{registered} \wedge c2 \neq c? \wedge \text{lease } c2 = \text{lapsed} \wedge \text{owner } s = \text{conn}(c2)\}$ $\times \{\text{unowned}\})) s? = \text{unowned}$ $\vee$ $(\text{owner} \oplus (\{s : \text{SCENE} \mid \exists c2 : \text{CONN} \bullet$ $c2 \in \text{registered} \wedge c2 \neq c? \wedge \text{lease } c2 = \text{lapsed} \wedge \text{owner } s = \text{conn}(c2)\}$ $\times \{\text{unowned}\})) s? = \text{conn}(c?)$ $\text{owner}' = (\text{owner} \oplus (\{s : \text{SCENE} \mid \exists c2 : \text{CONN} \bullet$ $c2 \in \text{registered} \wedge c2 \neq c? \wedge \text{lease } c2 = \text{lapsed} \wedge \text{owner } s = \text{conn}(c2)\}$ $\times \{\text{unowned}\})) \oplus \{s? \mapsto \text{conn}(c?)\}$ $\text{identity}' = \text{identity}$ $\text{transport}' = \text{transport}$

7 Verification: the goal that must flip

Against this variant, I2/I5's negation must be *found* — the trace in `connection_lease_reaping.tex`, Section H1, reaches it directly: `Connect(c1, i1); Claim(c1, s1); TransportDies(c1); LeaseLapse(c1); Read(c1)`.

```
# must be FOUND against this variant (NOT found against the fixed spec)
probccli docs/connection_lease_reaping_destructive_read_buggy.tex \
-model_check -nodead \
-goal "#s.(#c.(s:SCENE & c:CONN & owner(s) = conn(c) &
      not(c : registered)))" \
-p DEFAULT_SETSIZE 3
```

I1 must stay unaffected — this variant leaves `Depart` and `TransportDies` untouched, so the chain `TransportDies;Depart` still completes exactly as in the fixed spec:

```
# must remain FOUND -- this variant does not touch Depart or
# TransportDies, only what a read does to a lapsed connection's
# registration
probccli docs/connection_lease_reaping_destructive_read_buggy.tex \
-model_check -nodead \
-goal "#c.(c:CONN & c : dom(identity) & not(c : registered) &
```

```
        transport(c) = tdown)" \  
-p DEFAULT_SETSIZE 3
```